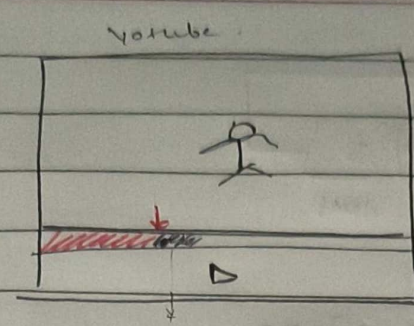
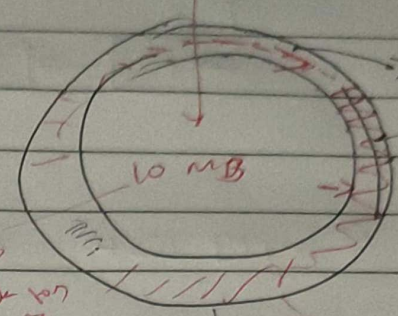


# 13 Circular List in Python



for 500 MB video

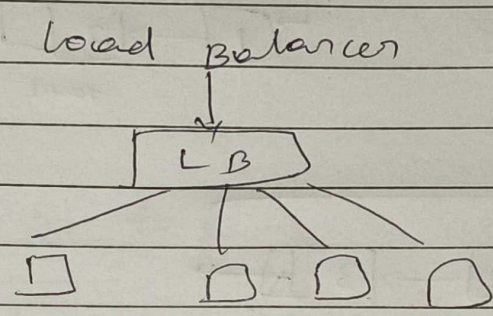


It comes back  
deleted  
fast  
forward load in  
next video  
but can't see

also  
work for  
rings

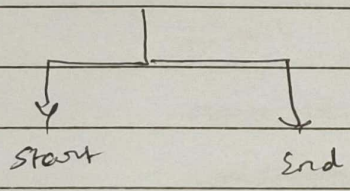
Circular linked list

Scheduling  
CPU → Round Robin

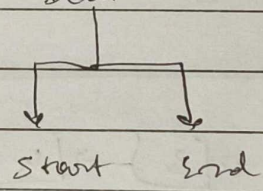


Other Linked list

→ Insert

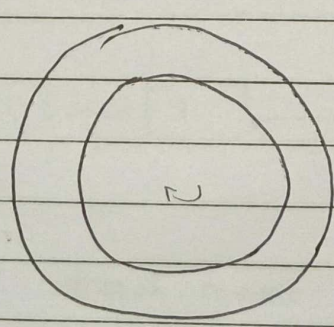


Delete

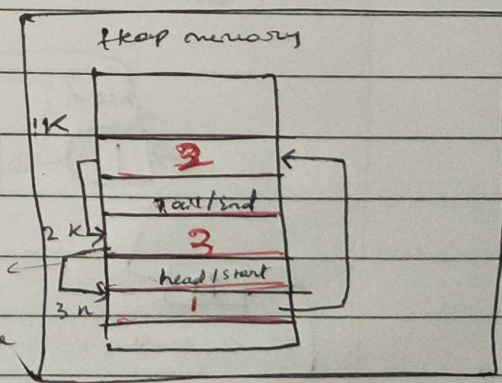


Search  
Point

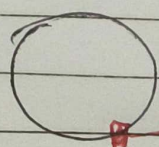
Circular linked list :-



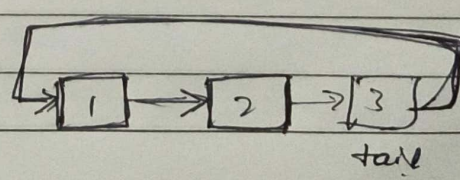
RAM



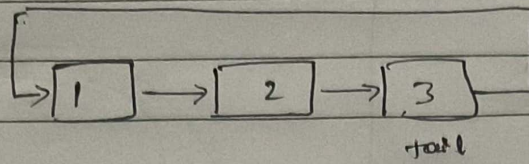
last  
node  
node  
point  
to None



→ Tail / End

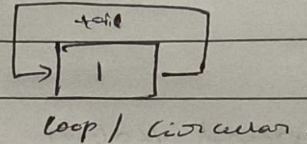




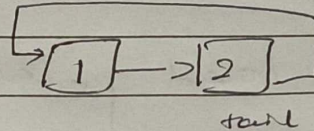


1) Empty list  $\text{tail} = \text{None}$

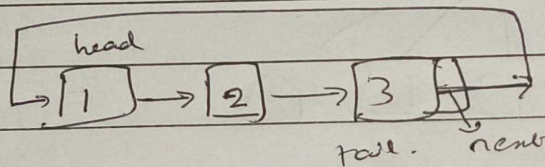
2) Single node



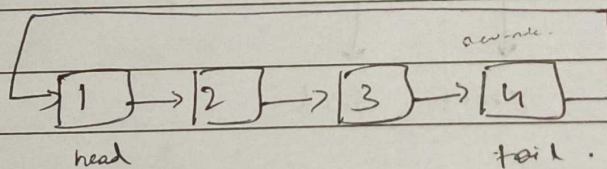
3) 2/3 Node



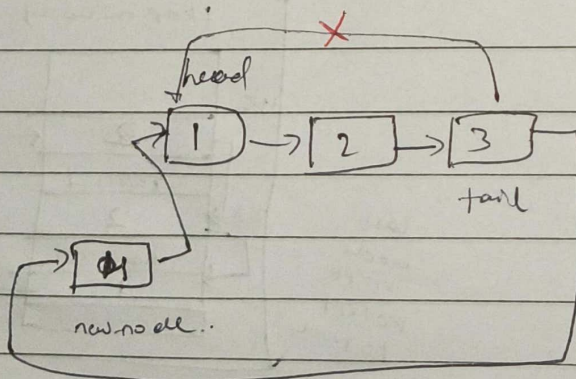
4)



\* Insert at end / tail

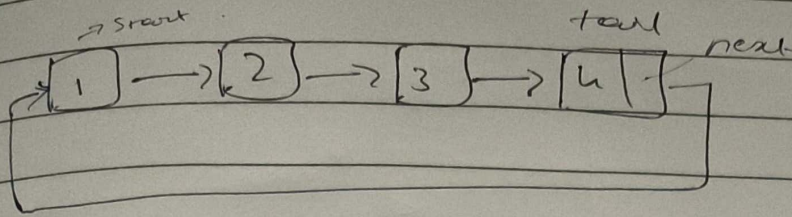


\* Insert at head / beginning





## Search / print

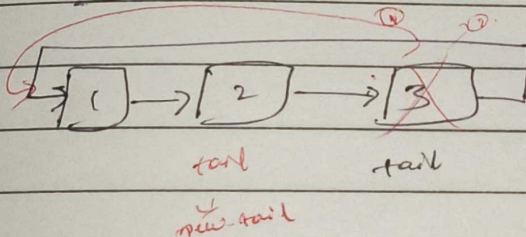


current-node = tail.next

current-node == tail

## ✶ Delete :

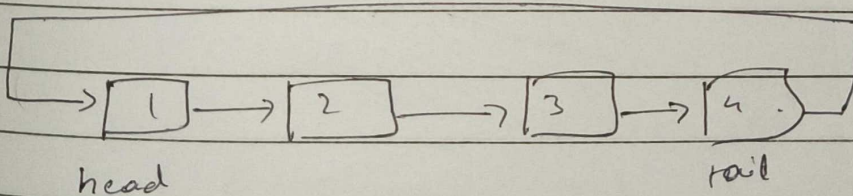
### Delete At tail



new-tail.next = tail.next

~~new-tail~~ tail = new-tail

### Delete At head



tail.next = tail.next.next



```
33 class Node:
34
35     def __init__(self, data):
36         self.data = data
37         self.next = None
38
39 class CircularList:
40
41     def __init__(self):
42         self.tail = None
43
44     # This function insert a node after tail (start)
45     # If there is node at start then it becomes the 2nd node
46     # Tail node remains the same
47     # Time complexity O(1)
48     def insert_at_beginning(self, data):
49         # Create new node
50         new_node = Node(data)
51
52         # Empty list
53         if (self.tail == None):
54             self.tail = new_node
55             new_node.next = new_node
56             return
57
58         # Single node / multi node
59         new_node.next = self.tail.next
60         self.tail.next = new_node
61
```



```

63 # This function insert node at tail and new node becomes the tail
64 # Time complexity O(1)
65 def insert_at_tail(self, data):
66     new_node = Node(data)
67
68     # Case 1: Empty list
69     if (self.tail == None):
70         self.tail = new_node
71         new_node.next = new_node
72         return
73
74     # You don't know the address of node before the tail node.
75     # Case 2: Single node / multi node
76     new_node.next = self.tail.next
77     self.tail.next = new_node
78     self.tail = new_node
79
80 # Delete the node after tail node
81 # Tail remains same
82 def delete_at_beginning(self):
83     # List empty
84     if (self.tail == None):
85         print("List is empty")
86         return
87
88     # Case 1: Single node then list will become empty
89     if (self.tail.next == self.tail):
90         self.tail = None
91         return
92
93     # Case 2: Two or more nodes
94     self.tail.next = self.tail.next.next
95

```



```
# we have to traverse to reach there
```

```
def delete_at_tail(self):  
    # Case 1: Empty list  
    if (self.tail == None):  
        return
```

```
    # Case 2: Single node  
    if (self.tail.next == self.tail):  
        self.tail = None  
        return
```

```
    # Case 3: 2 or more nodes  
    # Find the ref to the node previous to the current tail node  
    new_tail = self.tail.next  
    while (new_tail.next != self.tail):  
        new_tail = new_tail.next
```

```
    new_tail.next = self.tail.next  
    self.tail = new_tail
```

```
def print_all_nodes(self):  
    # Case 1: List is empty  
    if (self.tail == None):  
        print("List is empty")  
        return
```

```
    current_node = self.tail.next  
    while (True):  
        print(f" {current_node.data} --> ")  
  
        if (current_node == self.tail):  
            break
```

```
        current_node = current_node.next  
    # This is after the while loop
```



```
135 def search_key(self, key):
136     # Case 1: List is empty
137     if (self.tail == None):
138         print("List is empty")
139         return
140
141     current_node = self.tail.next
142     while (True):
143         if (current_node.data == key):
144             print("Key is found in the list")
145             return
146
147         if (current_node == self.tail):
148             break
149
150         current_node = current_node.next
151
152     print("Key is not present in the list")
153
```

```
154 def Circular_list_tests(clist: CircularList):
155     # List is empty and trying to delete a node
156     clist.delete_at_beginning()
157     clist.delete_at_tail()
158     clist.print_all_nodes()
159     clist.search_key(40)
160
161     # Perform insert and delete single node
162     clist.insert_at_beginning(10)
163     clist.delete_at_beginning()
164
165     clist.insert_at_tail(30)
166     clist.delete_at_tail()
167
168     # Create list with multiple nodes
```